

Planning

בשביל שיהיה לנו עץ שנוכל לבצע עליו חיפוש, אנחנו צריכים דרך להגדיר את הבעיה בצורה פורמלית. אם הבעיה לא תהיה מוגדרת היטב, העץ יהיה גדול יותר ממה שצריך. בעיה דטרמיניסטית בשביל planning מוגדרת ע"י $\langle V, I, A, G \rangle$:

V קבוצה סופית של משתני מצב

I מצב התחלתי של V

A פעולות על V

G נוסחה על V שמתארת מצבי מטרה

זוהי הגדרה מאוד אבסטרקטית. יש שתי שפות שמממשות אותה - STRIPS ו-FDR

STRIPS

ייצוג קלאסי שבו:

- מצב הוא אוסף של עובדות
- לפעולה יש דרישות קדם (איזה עובדות צריכות להיות נכונות) ואפקטים (איזה עובדות להוסיף ואיזה לגרוע מהמצב)

STRIPS נותן לנו:

- יוריסטיקות שימושיות
- הגדרה פורמלית של מושגי היסוד
- שפה - PDDL - שמאפשרת לקודד בעיות

הגדרה פורמלית:

$$\Pi = \langle P, O, I, G \rangle$$

P קבוצה סופית של אטומים - ליטרל שיכול להיות true או false.

$I \subseteq P$ מצב התחלתי - כל אטום $p \in I$ הוא true וכל אטום $p \notin I$ הוא false.

$G \subseteq P$ הגדרת מטרה - קבוצה של אטומים דרושים למטרה.

O אוסף פעולות (אופרטורים) $o = \langle pre(o), add(o), del(o) \rangle$, כאשר $pre(o), add(o), del(o) \subseteq P$.

γ מעבר (transition) $\gamma(s, o) = (s \setminus del(o)) \cup add(o)$

π תוכנית (plan) היא $\pi = \langle o_1, \dots, o_k \rangle$ כאשר $s_0 = I$, $s_i = \gamma(s_{i-1}, o_i)$ ו- $G \subseteq s_k$

FDR

הבעיה עם STRIPS היא שכל המשתנים הם בינארים. FDR מחליף את זה במשתנים:

$$\Pi = \langle V, D, O, I, G \rangle$$

משתנים	V
דומינים: לכל $v \in V$ מוגדר D_v , ו- $D = \{D_v v \in V\}$	D
מצב מוגדר $s \in \prod_{v \in V} D_v$	
אופקטורים $o = \langle pre(o), eff(o) \rangle$ כאשר:	O
$pre(o)$: דרישות קדם (השמה חלקית למשתנים)	
$eff(o)$: אפקטים (שינוי ערכים)	

דומינים

דומיין מגדיר איך העולם מיוצג ואיך הוא משתנה, בלי קשר לבעיה/משימה ספציפית. אנחנו רוצים למצוא פתרון שיהיה domain independent שיתאים להרבה בעיות - ואז נוכל לעשות יוריסטיקות ולתכנן מסלולים שיתאימו לSTRIPS או לFDR. קוראים למנוע הזה פתרון (planner), והוא:

- קורא את הבעיה, וממיר אותה לשפה שהוא מכיר.
 - בונה את משימת החיפוש.
 - מבצע חיפוש.
 - משתמש ביוריסטיקה כללית (שלא תלויה בבעיה ספציפית)
 - מחזיר תוכנית (plan) שפותרת את הבעיה.
- זו הכללה טובה שמאפשרת להשתמש באותו planner להרבה משימות שונות, אבל פחות טובה מאלגוריתם יעודי לבעיה ספציפית.
- PDDL שפה פורמלית לתיאור בעיות, שפתרנים ג'נרים יודעים להמיר לSTRIPS או לFDR ולפתור.
- יש כל מיני פתרנים עם תכונות שונות.

Domain-Dependant Planning

פתרון שיועד לפתור בעיה ספציפית נקרא Domain-Dependant Planner. פתרון כזה מגדיר successor function משל עצמו, שהיא יותר יעילה מסתם לנסות את כל הפעולות. זה מאפשר:

- לעשות pruning.
- למצוא יוריסטיקות יותר טובות שמשמשות בידע על הדומיין.
- להגדיר חוקים - פעולות שלא עושים למרות שהן חוקיות.
- להגדיר macros - סט של פעולות שצריכות לבוא אחת אחרי השנייה.
- לבחור את אלגוריתם החיפוש הכי מתאים (או להגדיר אלגוריתם מותאם אישית)

Delete-Relaxation

בעיית planning היא PSPACE-hard. נרצה להשתמש ביוריסטיקות כדי להפוך אותה ליותר קלה. דרך כללית לעשות את זה ב-STRIPS היא delete-relaxation. הגדרות:

- תהי $\Pi = \langle P, O, I, G \rangle$ משימת תכנון STRIPS כאשר כל $o \in O$ הוא $o = \langle pre(o), add(o), del(o) \rangle$.
- הגרסה delete-relaxed שלה היא $\Pi^+ = \langle P, O^+, I, G \rangle$ כאשר לכל $o \in O$ הגרסה המוחלשת שלה היא $o^+ = \langle pre(o), add(o), \emptyset \rangle$ ו- $O^+ = \{o^+ | o \in O\}$.
- h^+ היא אורך הפתרון ב- Π^+ עבור כל מצב ב- Π .
- h^+ היא יוריסטיקה אדמיסיבילית - אבל NP-קשה לחישוב.

RPG - Relax Planning Graph

את הגרף הזה אפשר לבנות בזמן פולינומי. בונים אותו בצורה הבאה:

- השכבה הראשונה היא המצב ההתחלתי $F_0 = I$. כל fact מסומן בעיגול - אם יש צבע זה אומר שהוא "דלוק" (יש לנו אותו)
- שכבת פעולה A_k היא כל הפעולות שאפשר להפעיל על F_k :
$$A_k = \{o | pre(o) \subseteq F_k\}$$
- נשים לב שזה כולל גם את הפעולות שכבר הופעלו - כי preconditions שלהם עדיין דולקות.
- שכבת fact היא כל הפעולות משכבת fact הקודמת, וגם האטומים ששכבת הפעולה הפעילה:

$$F_{k+1} = F_k \cup \bigcup_{o \in A_k} add(o)$$

• עוצרים כאשר $G \subseteq F_k$ או fixpoint : $F_{k+1} = F_k$.

לכל היותר יהיו לנו $\min\{|P|, |O|\}$ שכבות.
למה צריך את זה?

• אם $G \not\subseteq F_k$ ב fixpoint אזי הבעיה לא פתירה מ I .

• אפשר לחלץ מזה יוריסטיקות $h_{FF} \vee h_{add}, h_{max}$.

יוריסטיקות מתוך RPG

יוריסטיקת h_{max}

היוריסטיקה הזאת מסתכלת על כל goal כמטרה נפרדת, וסופרת מה המספר המינימלי של פעולות שצריך בשביל להשיג אותו. המקסימלי מבין הgoalים הוא h_{max}

1. לכל $p \in I$ נסמן:

$$\text{cost}(p) = 0$$

2. העלות של כל פעולה היא:

$$\text{cost}(o) = 1 + \max_{p \in \text{pre}(o)} \text{cost}(p)$$

(מחכים ל precondition הכי "איטי", ומוסיפים את עלות הפעולה 1)

3. העלות של fact היא:

$$\text{cost}(p) = \min_{o: p \in \text{add}(o)} \text{cost}(o)$$

(הפעולה הכי זולה שמספקת את p)

לאחר שמוצאים את כל המחירים, מחשבים:

$$h_{max}(s) = \max_{g \in G} \text{cost}(g)$$

בפועל לא בונים את הגרף ואז מחשבים, אלא בונים אותו תוך כדי תנועה. בכל פעם שמפעילים פעולה או fact אפשר למצוא לו את המחיר - מחיר שלא ישתנה.

יוריסטיקת h_{add}

דומה ל h_{max} , אבל חישוב העלות של פעולה הוא לפי סכום העלויות של preconditions שלה:

$$\text{cost}(o) = 1 + \sum_{p \in \text{pre}(o)} \text{cost}(p)$$

$cost(p)$ נשאר אותו דבר.
סוכמים גם בחישוב הסופי:

$$h_{add}(s) = \sum_{g \in G} cost(g)$$

היוריסטיקה הזאת אינה אדמיסיבילית, אבל היא עדיין שימושית שכן היא חוסמת את $h^+(s)$ מלמעלה:

$$h^+(s) \leq h_{add}(s)$$

יוריסטיקת h_{FF}

h_{FF} היא חסם עליון יותר הדוק:

$$h^+(s) \leq h_{FF}(s) \leq h_{add}(s)$$

הרעיון הוא למצוא תוכנית כלשהי s ב G ו Π^+ ולספור את מספר הפעולות בה. בשביל היוריסטיקה הזאת (בניגוד לקודמות) צריך קודם כל לבנות את הגרף ורק אז אפשר לחשב את h_{FF} .
מחפשים "לספק" את כל הצמתים מה $goal$ עד למצב ההתחלתי, כאשר בכל פעם מחפשים קודקוד מסומן אבל לא מסופק ומסמנים קודקוד שקדם לו לפי חוקים (החוק הראשון עבורו מוצאים קודקוד מתאים):

1. אם יש קודקוד פעולה מסומן לא מסופק - מסמנים את כל pre שלו.
 2. אם יש קודקוד $fact$ לא מסופק עם פעולה אחת בלבד שמפעילה אותו - מסמנים את הפעולה.
 3. אם יש קודקוד $fact$ שסופק בשכבה קודמת - מסמנים אותו בשכבה הקודמת.
 4. אם יש קודקוד $fact$ עם כמה פעולות שמפעילות אותו - בוחרים שרירותית (או עם $tie-breaker$) את אחת הפעולות שמפעילות אותו, ומסמנים אותה.
- הפעולות המסומנות הן התוכנית.